Task 3: Convolutional Neural Networks:

Convolutional Neural Networks are a type of neural network designed for spatially structured data, specifically images. Their main use is in computer vision tasks such as image classification, object detection, semantic segmentation, and facial recognition. CNNs are effective because they preserve spatial relationships while using local connectivity and weight sharing, which greatly reduce the number of parameters compared to fully connected networks. As a result, they can learn visual patterns such as edges, textures, shapes and other semantic features.

Convolutional Neural Networks Theory:

Foundation of CNNs:

- Spatially Structured data: Data in which the position of each value matters. In images, a pixel's meaning depends not only on its value but also on where it is located relative to nearby pixels.
- Computer Vision: A field of artificial intelligence focused on enabling computers to interpret, analyze, and make decisions from images or videos.
- Image classification: A task where a model assigns a single label or category to an entire image, such as "cat," "car," or "dog."
- Object detection: A task where a model identifies both what objects are present and where they are located, usually by drawing bounding boxes around them.
- Semantic Segmentation: A task where every pixel in an image is assigned a class label, such as road, sky, person, or building.
- Facial Recognition: A computer vision application that identifies or verifies a person based on facial features extracted from an image.
- Convolution: The core operation of a CNN in which a small set of learnable weights scans across the input to detect local patterns such as edges, corners, and textures.
- Local connectivity: A CNN design principle where each neuron only looks at a small local region of the input instead of the entire image, which helps preserve spatial structure.
- Weight sharing: The use of the same filter weights across many spatial locations in the input. This greatly reduces the number of parameters and lets the model detect the same pattern anywhere.
- 1-D settings: Applications where the data is arranged along one dimension instead of two, such as time signals, sound waves, or text sequences.
- Time-Series classification: A task where sequential data measured over time is classified into categories, such as identifying normal vs abnormal sensor behavior.
- Text classification: A task where text is assigned to categories, such as spam detection, sentiment analysis, or topic labeling.
- Fully Connected (FC) layer: A layer in which every input neuron connects to every output neuron. It is powerful but expensive in parameters and ignores spatial layout.
- Parameters: The trainable numerical values of a model, mainly weights and biases, that are adjusted during training.
- Ignoring spatial relationships: A weakness of FC layers on images: they treat pixels as a flat list and lose

information about which pixels are near one another.

- Kernel: A small matrix of weights, such as 3×3, used to scan over an input and compute local responses.
- Translation equivariance: A property of convolutions where shifting a pattern in the input causes a corresponding shift in the output feature map, rather than destroying detection.
- Hierarchy of abstractions: The layered learning process of CNNs, where early layers learn simple features and deeper layers learn more complex concepts.
- Edges: Basic visual features where there is a sharp change in brightness or color, often forming boundaries of objects.
- Colour Gradients: Smooth changes in color or intensity across a region of an image.
- Textures: Repeated local visual patterns, such as fur, grass, or fabric surfaces.
- Semantic concepts: High-level meaningful features such as faces, wheels, or entire objects.
- Visual cortex: The part of the brain that processes visual information. CNNs are often said to loosely mimic its hierarchical pattern processing.
- Oriented edges: Edges that point in specific directions, such as horizontal, vertical, or diagonal.
- Blobs: Small connected regions of similar intensity or color that can act as primitive visual features.
- Object Parts: Intermediate-level features such as eyes, wheels, ears, or handles that help form larger object understanding.
- Discriminative feature: A feature that strongly helps distinguish one class from another.
- Class probabilities: The predicted probabilities assigned by a model to each possible category.
- Spatial Generalisation: The ability of a CNN to recognize patterns even when they appear at different image positions.
- Pooling: A downsampling operation that summarizes local neighborhoods in a feature map, reducing size and computation.
- Translation Invariance: The property that small shifts in the input do not significantly change the final prediction.
- Robust: Able to continue performing well despite noise, variation, or small changes in the input.
- Positional Shifts: Small changes in where a pattern appears in the image.

Kernels, Filters & Convolution:

- Patch: A small region of the input or feature map that the kernel covers at one time.
- Overlapping Input variable: The specific input number that lies directly under a kernel weight during the convolution calculation.
- Top-Left patch: The first patch processed when the kernel begins scanning from the upper-left corner of the input.
- Output position: The location in the output feature map where the computed result for one kernel placement is stored.
- Kernel Indices: The row and column positions used to refer to entries inside the kernel matrix.
- Cross Correlation: The actual operation typically used in deep learning, where the kernel slides over the input without being flipped. It is commonly called convolution in practice.

Output Size, Stride, Padding & Kernel Size:

- Output Size Formula: Formula used to calculate the spatial size of the output after convolution:
  - $Output = \frac{W - K + 2P}{S} + 1$
  - W = Spatial Size (height or width of the input)
  - K = Kernel Size
  - P = Padding
  - S = Stride (how many pos. A kernel moves per step)
- Hyperparameter: A setting chosen before training, such as kernel size, stride, padding, learning rate, or number of layers.
- Receptive field: The region of the input that influences a particular output neuron.
- Coarser Patterns: Larger and less detailed patterns captured by bigger kernels or deeper layers.
- Dense Output: An output produced with closely spaced computations, often when stride is 1.
- Spatial Dimensions: The height and width of an input or feature map.
- Learnable Downsampling: Reducing spatial size using operations with trainable weights, such as strided convolution.
- Border pixels: Extra pixels added around the edges of the input, usually zeros.
- Same Padding: Padding chosen so the output keeps the same spatial size as the input, for odd kernels:
  - $P = \frac{K - 1}{2}$
- Valid Padding: No added Padding is used:
  - P = 0
- Output shrink: The decrease in output size caused by convolution without enough padding.
- Spatial Collapse: When repeated shrinking causes the feature map to become too small to remain useful.

Feature Maps, Filters & Channels:

- Activation: The numerical output value produced by a neuron or filter response.
- High activation: A large value indicating the filter strongly detected its target pattern.
- Strong match: A close correspondence between the filter's learned pattern and the local input region.
- Specialising through training: The process by which different filters gradually become tuned to different patterns during learning.
- *Typical layers use 32-512 layers*: A common range for the number of filters in CNN layers, increasing with network depth and task complexity.
- Tensor: A multi-dimensional array of numbers. In CNNs, inputs, outputs, and weights are usually represented as tensors.
- Output volume: The full 3-D output of a convolutional layer, consisting of height, width, and depth.
- Formula for output volumes: If a layer uses F filters, the the output depth is F, so the output volume is:
  - $H_{out} + W_{out} + F$
  - H_{out} = output weight
  - W_{out} = output width
  - F = number of filters

Non-Linearity & Activation Functions:

- Linear transformation: A mathematical mapping that combines inputs through weighted sums without introducing complex curved behavior.
- Arbitrary decision boundary complexity: The ability of a neural network to model very simple or very complicated separations between classes.

- Universal function approximator: A model that can approximate a wide range of functions if it has enough hidden units and non-linearity.
- Matrix Multiplication: A basic linear algebra operation used in neural networks to combine inputs and weights.
- Decision Boundary: The boundary a model creates in feature space to separate one class from another.
- Straight-Line decision boundaries: Linear class separations that cannot solve many real-world problems with complex structure.
- Non-Linearly separable problem: A classification problem where classes cannot be separated by a single straight line or flat hyperplane.
- Sigmoid: Activation function which squashes values between 0 and 1:
  - $\sigma(x) = \frac{1}{1+e^{-x}}$
- Vanishing gradient: A training problem where gradients become extremely small as they are propagated backward, causing early layers to learn very slowly.
- Extremes: Very large positive or very large negative values, where some activation functions flatten out and produce tiny gradients.
- Tanh: activation function which squashes values between -1 and 1:
  - $tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$
- Centred: Having outputs centered around zero, which can sometimes improve optimization.
- ReLU: Activation function which outputs 0 for negative inputs and keeps positive inputs unchanged:
  - ReLU(x) = max(0,x)
- Dying Neuron Problem:A problem where ReLU neurons output zero for all inputs and stop updating because they remain in the negative region.
- Leaky ReLU: variation which allows small negative output instead of zero for negative inputs:
  - Leaky ReLU(x) = max(0.01x, x)
- Negative Slope: The small nonzero multiplier applied to negative inputs in Leaky ReLU.
- Softmax: Activation function used in the final classification layer to convert outputs into probabilities that add up to 1:
  - $\text{Softmax}(x_i) = \frac{e^{x} - e^{-x}}{\sum_j e^{x} j}$
- Binary Output: A prediction setup with two possible classes.
- Multi-class output layer: A final layer that predicts among more than two classes.
- Gradient 1: For positive ReLU inputs, the derivative is 1, which helps prevent vanishing gradients in that region.
- Computationally trivial: Very simple and fast for a computer to compute.
- PReLU: Parametric ReLU, a variation where the negative slope is learned instead of fixed.
- Multinomial logistic regression: A classification method for multiple classes; Softmax is closely related to it.

Pooling Layers:

- Downsamples / Downsampling: Reducing the height and width of a feature map.
- **Translation Invariance:**
- Overfitting: When a model performs very well on training data but poorly on unseen data because it memorized details rather than learned general patterns.

- Max pooling: A pooling method that takes the largest value in each local region.
- Average pooling: A pooling method that takes the average value in each local region.
- Window: The local region over which pooling is computed.
- Background context: Surrounding, less dominant information in an image that may still help interpretation.
- Global Average Pooling (GAP): A pooling operation that averages each entire feature map into one number.
- Classifier Head: The final part of the network that converts extracted features into class predictions.
- Fixed rule: An operation that behaves the same every time and does not learn from data.
- Small input shifts and same output: A shorthand description of translation invariance.

Building a CNN - Architecture Pipeline:

- Inference: The stage where the trained network is used to make predictions on new inputs.
- Forward Pass: The movement of data through the network from input to output.
- Gradients: Values showing how much the loss changes with respect to each parameter.
- Backpropagation: The algorithm that computes gradients by propagating error backward through the layers.
- Batch Normalization: A layer that normalizes activations across a mini-batch to improve training stability and speed.
- Normalises activation: Adjusts activation values so they follow a more controlled distribution during training.
- Stabilise training: Reducing wild changes or instability during optimization.
- Flatten: Reshaping a multi-dimensional feature tensor into a 1-D vector.
- 1-D vector: A one-dimensional list of numbers.
- Class logits: Raw scores output by the final linear layer before Softmax converts them into probabilities.

Training CNNs:

- Ground-truth Label: The correct answer provided in the dataset.
- Loss function: A function that measures how wrong the model's predictions are.
- Chain Rule: A calculus principle used during backpropagation to compute gradients through compositions of functions.
- Optimizer: The algorithm that updates model parameters using gradients.
- Cross-Entropy Loss: A standard loss function for classification that heavily penalizes confident wrong predictions.
- Penalization of confident wrong predictions: Giving a large loss when the model predicts the wrong class with high confidence.
- Mean Squared Error (MSE): Loss function for regression that averages the squared differences between predicted and true values:
  - $\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- Regression: A task where the model predicts a continuous number rather than a class.
- Outliers: Unusually extreme data points that can disproportionately affect training.
- Focal Loss: A modified classification loss that reduces the importance of easy examples and focuses learning on harder ones.

- Down-weights: Assigns lower influence or smaller contribution.
- Upstream gradient: The gradient arriving from later layers during backpropagation.
- Fully differentiable: Every part of the operation has a derivative, allowing gradient-based training.
- Gradient descent: An optimization method that updates parameters in the direction that reduces loss.
- Adaptive Learning Rate: A learning-rate strategy where different parameters can effectively receive different step sizes.
- Momentum: A method that smooths updates by accumulating past gradient direction.
- Stochastic Gradient Descent (SGD): A simple optimizer that updates parameters using gradients estimated from data batches.
- Generalisation: How well a trained model performs on new unseen data.
- RMSProp: An optimizer that scales updates using a moving average of squared gradients.
- RNNs: Recurrent Neural Networks, commonly used for sequential data.
- Non-Stationary objectives: Optimization settings where the effective target behavior changes over time.
- Adam: An optimizer that combines adaptive learning rates with momentum-like estimates.
- AdamW: A corrected version of Adam that handles weight decay more properly.
- Transformers: A neural network architecture originally designed for sequence modeling, now also widely used in vision.
- First-moment estimate: The moving average of gradients in Adam, related to momentum.
- Second-moment estimate: The moving average of squared gradients in Adam, used to adapt step sizes.
- Bias correction term: A correction in Adam that compensates for initial underestimation in moving averages.
- Weight decay: A regularization technique that discourages weights from becoming too large.
- Transformer-based models: Models built using Transformer layers or principles.

Regularisation, Dropout & Batch Normalisation:

- Regularisation:Any method used to reduce overfitting and improve generalization.
- Memorise training data: When a model learns exact training examples rather than transferable patterns.
- Generalisable patterns: Patterns that remain useful on unseen data.
- Model complexity: How expressive or flexible a model is, often related to depth, width, and parameter count.
- Controlled noise: Deliberate randomness added during training to make learning more robust.
- Dropout: A regularization method that randomly sets some activations to zero during training.
- Fraction p*:* The probability or proportion of units dropped by dropout.
- Redundant representations: Multiple neurons learning overlapping useful information so the model does not rely on one specific neuron.
- Inference scaling by (1 - p): At inference, dropout is disabled, and outputs are scaled appropriately so expected activation magnitudes stay consistent.
- Data augmentation: Artificially expanding training diversity by transforming images.

- Effective training set size: The practical diversity of examples the model sees, even if the original dataset size stays the same.
- Realistic transforms: Changes like cropping or flipping that keep the image plausible and label-consistent.
- Flips: Horizontal or vertical mirroring of an image.
- Random Crops: Randomly selected subregions of an image used during training.
- Colour jitter: Random changes to brightness, contrast, saturation, or hue.
- Rotation: Turning the image by some angle.
- Cutout: Masking or removing a region of the image during training.
- Mixup: A technique that combines two inputs and their labels to regularize training.
- Invariant Representation: A learned feature representation that stays useful even when the input changes slightly.
- Mini-batch: A small subset of training data processed together in one optimization step.
- Unit variance: A normalized variance equal to 1.
- Learned scale γ: A trainable BatchNorm parameter that rescales normalized activations.
- Learned shift β: A trainable BatchNorm parameter that shifts normalized activations.
- Initialisation sensitivity: How much training quality depends on the model's starting weights.
- Batch Normalisation formula:
    - $BN(x) = \gamma \bullet \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$
    - x = input activation
    - μ = batch mean
    - σ^2 = batch variance
    - ε = small constant for numerical stability
    - γ = learned scale
    - B = learned shift
- Numerical stability: Preventing unstable calculations such as division by zero or overflow.
- Identify the technique: A prompt asking you to determine which regularization method is being described.

Famous CNN Architectures:

- Convolutional to Pooling to FC paradigm: A classic CNN design in which convolution extracts features, pooling reduces spatial size, and fully connected layers make the final prediction.
- Digit recognition: Recognition of handwritten or printed numbers.
- GPU: A graphics processing unit that accelerates parallel computations used in deep learning.
- ImageNet: A large benchmark dataset central to the development of modern image classification models.
- Transfer Learning backbone: A pretrained network reused as the feature extractor for a new task.
- Skip connection: A connection that lets input bypass one or more layers and be added later.
- ResNet skip formula:
    - Output = $F(x, \{W_i\}) + x$
    - x = input to block
    - $F(x, \{W_i\})$ = learned residual mapping
    - $\{W_i\}$ = weights in the block
- Residual mapping: The part $F(x, \{W_i\})$that the network learns in a residual block.
- Identity shortcut: The direct path that carries the input *x* unchanged to the output addition.

- Train stably: To learn without severe optimization failure, such as vanishing or exploding gradients.
- Depthwise separable convolution: A cheaper convolution method that splits standard convolution into depthwise and pointwise steps.
- Depthwise convolution: Applying one spatial filter per input channel independently.
- Pointwise mixing: Using 1×1 convolutions to mix information across channels after depthwise convolution.
- 1 x 1 pointwise convolution: A convolution with kernel size 1 used to combine channel information.
- Mobile deployment: Running a model efficiently on phones or low-power devices.
- Transformer design choices: Architectural decisions inspired by Transformers, such as normalization placement or larger kernels in some modern CNN redesigns.
- Normalization layer: A layer that rescales activations to improve optimization behavior.
- Vision Transformer (ViT): A Transformer-based model adapted for image data.
- Pure CNN: A model built entirely from convolutional ideas without Transformer layers.
- Minimal Accuracy Loss: Only a small drop in performance despite major reductions in computation or model size.

Designing CNNs & Diagnosing Problems:

- Backbone: The main pretrained feature extractor used in a model.
- Pre-trained weights: Weights learned previously on a large dataset and reused as a starting point.
- Fine-tune: Continue training a pretrained model on a new dataset so it adapts to the new task.
- FC head: The final fully connected prediction component attached after the backbone.
- Bounding box regression: Predicting the numerical coordinates of an object's box.
- Feature Pyramid Network (FPN): A multi-scale feature architecture that combines information from several depths to detect objects of different sizes.
- Multiple scales: Different object sizes present in images.
- YOLO: A family of one-stage object detectors that predicts boxes and classes in one forward pass.
- Single-shot approach: Making all detection predictions at once rather than using separate proposal and refinement stages.
- Faster R-CNN: A two-stage detector that first proposes candidate object regions and then classifies and refines them.
- Two-stage region proposal method: An object detection approach with a region-proposal stage followed by classification/refinement.
- Validation accuracy: Accuracy measured on a separate validation set used to estimate generalization during training.
- Early stopping: Stopping training when validation performance stops improving, to avoid overfitting.
- Insufficient capacity: A model being too small or too simple to learn the task well.
- Epochs: Complete passes through the full training dataset.
- Tuned learning rate: Complete passes through the full training dataset.

- First-Layer weights: The filters learned in the first convolutional layer.
- Tensor Shape: dimensions of convolutional weights: number of filters, number of channels, kernel height, and kernel width.
    - F C K K
    - F = number of filters.
    - C = number of input channels.
    - K = Kernel height and width
- Pytorch: A deep learning framework widely used for research and model development.
- Matplotlib: A Python plotting library used to display graphs and images.
- Diverging Colourmap: A color scale designed to show deviations above and below a midpoint with contrasting colors.
- RdBu: A common red-blue diverging colormap.

Vanishing Gradient Problem:

- Derivative info: The slope values contributed by each layer during backpropagation.
- Shrinks exponentially with depth: Repeated multiplication by values less than 1 causes the gradient to decrease very rapidly as depth increases.
- Near-zero gradients: Gradients so small that parameter updates become almost ineffective.
- Effectively shallow: A deep network behaving as though it were shallow because early layers fail to learn.
- He Initialization:A weight initialization scheme designed for ReLU network that preserves variance better.
- Variance-preserving initialization: An initialization method intended to keep activations and gradients from becoming too large or too small across layers.
- Gradient Highway: An informal term for a path, such as a residual connection, that lets gradients flow backward more easily.
- Exploding Gradients: A problem where gradients become extremely large during backpropagation, causing unstable updates.
- Gradient clipping: A technique that limits gradient magnitude to prevent exploding gradients.
- Gradients: if each layer contributes a derivative which is smaller than 1, then across many layers the total gradient behaves like a repeated multiplication:
    - $\prod_{l=1}^{L} f'$, if $f_l^{'} < 1$, the product is extremely small
- He initialisation formula: common for ReLU-based networks to initialize weights with variance:
    - Var(W) = $\frac{2}{n_{in}}$, n_{in} is the number of input connections to a layer.

Tidbits:

1. A 32×32 greyscale image has 1,024 pixels. If the first FC layer has 512 neurons, how many weights does it have?
   If the first FC layer has 512 neurons, and a grayscale image has 1024 inputs then 1024 x 512, it means there would be 524,288 weights.
2. A Conv layer with one 3×3 filter on the same image — how many weights does the filter itself have?
   One filter with 3 x 3 would have 9 weights.
3. In one sentence: why does the CNN number stay the same no matter how large the image gets?
   The CNN number stays the same because a convolution filter has a fixed kernel size, so its weights depend on the filter dimensions and input channels and not on the full image size.

4. W=64, K=5, P=0, S=1 → what is the output size?
   $\frac{64-5+0}{1} + 1 = 60$, the output size is 60.
5. W=64, K=3, P=1, S=1 → what is the output size? (Should equal input.)
   $\frac{64-3+2}{1} + 1 = 64$, the output size is 64.
6. W=64, K=3, P=0, S=2 → what is the output size?
   $\frac{64-3+0}{2} + 1 = 31.5$, so the output size would be 31.
7. What padding P keeps W=128 unchanged with K=5, S=1?
   To keep W = 128 unchanged with the current values the padding required would be P = 2.
8. A Conv layer has 32 filters, kernel size 3×3, input channels 3. How many learnable weights does it have? (Ignore bias.)
   Each filter has 3 x 3 x 3 = 27 weights per filter, meaning a total amount of learnable weights 32 x 27 = 864 total learnable weights.
9. Why can't a 10-layer network with no activations classify the XOR problem?
   A 10-layer network with no activations cannot classify XOR because all stacked linear layers collapse into one linear transformation, and XOR is not linearly separable
10. If the next layer has 64 filters with 3×3 kernels — how many weights now?
    Each filter has 3 x 3 x 32, meaning the total weights are 64 x 288 meaning 18, 432 total weights.
11. Why does adding more filters increase depth but not spatial size?
    Adding more filters increases depth because each filter produces one new feature map, but it does not change spatial size because height and width are controlled by kernel size, stride and padding.
12. Why does Cross-Entropy penalise a confident wrong prediction more than an uncertain wrong one?
    Cross-entropy penalizes a confident more because assigning a very high probability to the wrong class makes the loss much larger than being wrong, but uncertain.
13. Adam adapts the learning rate per parameter — give one situation where this is better than plain SGD.
    Adam is better than plain SGD when different parameters need different update sizes, such as in deep networks where some layers learn quickly and others have sparse or unstable gradients.
14. You randomly zero out 50% of neurons during training but not at test time. What technique is this?
    The technique is dropout.
15. Your model gets 99% train accuracy and 62% validation accuracy. Name the problem and list two fixes.
    This data indicates that the model is overfitting, two fixes may be stronger data augmentation and more regularization.
16. Look up the ImageNet top-1 accuracy and parameter count for AlexNet, VGG-16, ResNet-50, and MobileNetV2. Fill in a table.

Model	ImageNet Top-1 Accuracy	Parameters
AlexNet	~56.52%	61.1M
VGG-16	~71.59%	138.4M
ResNet-50	~76.00%	25.6M
MobileNetV2	~71.80%	3.5M

17. In your own words: why does adding more layers hurt accuracy in a plain network but help in ResNet?
    Adding more layers hurts a plain network because network optimization becomes harder and gradients can vanish, while ResNet helps because skip connections let information and gradients pass through more easily.
18. Epoch 1→20: train loss drops from 1.8 to 0.1; val loss drops to 0.4 then rises to 1.2. What is happening?
    If train loss drops from 1.8 to 0.1 while validation loss drops to 0.4 and then rises to 1.2, the model is overfitting after the point where validation loss stops improving.
19. Your model scores 55% on a 2-class problem (chance = 50%). Is it overfitting or underfitting?
    If the model scores 55% on a 2-class problem, where chance is 50% it is likely that is underfitting or the model isn't understanding anything useful.
20. If each layer multiplies the gradient by 0.5, what is the gradient magnitude after 10 layers? After 20?
    If each layer multiplies the gradient by 0.5, after 10 layers it will be approximately 0.0009765625 and after 20 0.0000009537.
21. Why does ReLU not vanish for positive inputs? Write the derivative of max(0,x) for x>0 and x<0.
22. ReLU does not vanish for positive inputs because the derivative is constant there, indicating that for positive inputs the gradient is 1.
23. You remove all pooling from a VGG-16 trained on 224×224 images. Estimate the spatial size of the final feature map before the FC layer.
    VGG-16 on 224 x 224 uses 5 pooling layers, each halving the spatial space 224 to 112 to 56 to 28 to 14 to 7. Meaning the final feature map is 7 x 7 with a depth of 512, with a final form of: 7 x 7 x 512.
24. You keep pooling but remove all ReLUs. In one sentence, what does the network become?
    If you keep pooling but remove all ReLUs, the whole network becomes one linear system with pooling between layers, so it loses the ability to learn complex non-linear decision boundaries.
25. Add BatchNorm after each Conv layer in the table above. Where exactly does it go (before or after ReLU)?
    The BatchNorm goes after Conv and before ReLU.
26. Replace both MaxPool layers with a strided Conv (stride=2). What do you need to change about the Conv config?
    If both MaxPool layers are replaced by strident Conv with stride 2, you need to change the convolution configuration so those conv layers use: stride 2, kernel size 3 x 3 and padding 1 to control the spatial reduction. So the MaxPool should look like this:

MaxPool(2 x 2, s = 2) to Conv(3 x 3, s= 2, P = 1)

Tidbit Math:

1. Compute One Output cell by hand:
    Matrix X:

$$\begin{bmatrix} 2 & 5 & 1 & 4 & 3 \\ 7 & 0 & 6 & 2 & 8 \\ 1 & 9 & 3 & 5 & 0 \\ 4 & 2 & 7 & 1 & 6 \\ 8 & 3 & 5 & 9 & 2 \end{bmatrix}$$

    Kernel K:

$$\begin{bmatrix} 1 & -1 & 2 \\ 0 & 2 & -2 \\ -1 & 1 & 0 \end{bmatrix}$$

    Process:
    (2 • 1) + (5 • -1) + (1 • 2) + (7 • 0) +

$(0 \bullet 2) + (6 \bullet -2) + (1 \bullet -1) + (9 \bullet 1) + (3 \bullet 0) = -5$

The output indicates the kernel's response to the first 3 x 3 path is negative.

Cell (1,1):

$(0 \bullet 1) + (6 \bullet -1) + (2 \bullet 2) + (9 \bullet 0) + (3 \bullet 2) + (5 \bullet -2) + (2 \bullet -1) + (7 \bullet 1) + (1 \bullet 0) = -1$

The output indicates that the kernel also gives slightly negative response at the center patch.

Feature Map (Result of sliding the kernel across the whole image):

$$\begin{bmatrix} -5 & 14 & -7 \\ 29 & -1 & 24 \\ 17 & 30 & -8 \end{bmatrix}$$

2. Edge Detectors:

Vertical Edge Filter:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$(2 \bullet -1) + (5 \bullet 0) + (1 \bullet 1) + (7 \bullet -2) + (0 \bullet 0) + (6 \bullet 2) + (1 \bullet -1) + (9 \bullet 0) + (3 \bullet 1) = -1$

The output indicates that the top left patch has a weak vertical edge response.

Horizontal Edge Filter:

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

$(2 \bullet -1) + (5 \bullet -2) + (1 \bullet -1) + (7 \bullet 0) + (0 \bullet 0) + (6 \bullet 0) + (1 \bullet 1) + (9 \bullet 2) + (3 \bullet 1) = 9$

The output indicates that the patch has a stronger horizontal edge response.

3. Activation Intuition:

ReLU

$ReLU(x) = \max(0,x)$

If $x < 0$, output = 0

If $x > 0$, output is $y = x$

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

If 0, x is negative

If 1, x is positive

4. Sketch the Pipeline

Stage	Layer / Operation	Settings	Output size	Learnable
1	Input	RGB image	32 x 32 x 3	No
2	Conv1	Kernel 3 x 3, 16 filters, stride 1, same padding	32 x 32 x 16	Yes
3	ReLU	Activation	32 x 32 x 16	No
4	MaxPool 1	2 x 2, stride 2	16 x 16 x 16	No
5	Conv2	Kernel 3 x 3, 32 filters, stride 1, same padding	16 x 16 x 32	Yes
6	ReLU	activation	16 x 16 x 32	No
7	Maxpool 2	2 x 2, stride 2	8 x 8 x 32	No
8	Flatten	reshape	$8 \bullet 8 \bullet 32 = 2048$	No
9	FC	2048 to 64	64	Yes
10	Output	64 to 10	10	Yes

		classes		

5. Pooling by hand:
   Top left 4 x 4 region:

$$\begin{bmatrix} 2 & 5 & 1 & 4 \\ 7 & 0 & 6 & 2 \\ 1 & 9 & 3 & 5 \\ 4 & 2 & 7 & 1 \end{bmatrix}$$

Max Pooling:

max $\begin{bmatrix} 2 & 5 \\ 7 & 0 \end{bmatrix}$
= 7

max $\begin{bmatrix} 1 & 4 \\ 6 & 2 \end{bmatrix}$
= 6

max $\begin{bmatrix} 1 & 9 \\ 4 & 2 \end{bmatrix}$
= 9

max $\begin{bmatrix} 3 & 5 \\ 7 & 1 \end{bmatrix}$
= 7

Result:

$$\begin{bmatrix} 7 & 6 \\ 9 & 7 \end{bmatrix}$$

Average pooling:

$$\frac{2+5+7+0}{4} = 3.5$$

$$\frac{1+4+6+2}{4} = 3.25$$

$$\frac{1+9+4+2}{4} = 4$$

$$\frac{3+5+7+1}{4} = 4$$

Result:

$$\begin{bmatrix} 3.5 & 3.25 \\ 4 & 4 \end{bmatrix}$$

6. Understand the Architecture:

Stage	Role
1) Input	Original image enters the network
2) Backbone / Encoder	Extracts features and compresses the spatial representation
3) ASPP	Captures multi-scale context utilizing diluted convolutions
4) Decoder	Restores spatial detail and sharpens boundaries
5) Upsample	Increases spatial reasoning
6) Pixel output	Predicts one class label per pixel

Paper Questions:

1. What problem is the paper solving? Why is vegetation detection difficult?
   The paper studies vegetation detection in high resolution UAV imagery, comparing deep learning methods against conventional NDVI methods for detecting vegetation. THe task is difficult because performance drops when the training and test images differ in appearance, resolution, or camera characteristics, and because deep learning methods need a lot of matching training to generalize data well. THe authors note that adding NIR information is useful, but models are limited in how many channels they can accept.
2. What deep learning models are evaluated? List each with its architecture type.
   a. DeepLabV3+: an encoder decoder semantic segmentation network with atrous spatial pyramid pooling and an Xception 65 backbone in their setup.
   b. Customized CNN: a patch based CNN classifier with four convolutional layers and one fully connected layer. Taking 7 x 7 x N patches and input and classifying them.
3. What is object-based detection and how does it differ from pixel-wise segmentation?
   In this paper, object-based detection means finding vegetation pixels with NDVI, and then grouping connected pixels into objects or connected components, and classifying those objects using rules, RGB statistics and ML steps like GMMs. This differs from pixel wise segmentation, where a model like DeepLabV3+ predicts a class for each pixel directly from the image in an ent to end way. So object based detection works on grouped regions, while pixel wise segmentation labels pixels individually.
4. How is DeepLabV3+ applied here — what input data does it use (RGB / multispectral / NDVI)?
   DeelLabV3+ is used in ways, with standard RGB images and with NDVI-GB input, where the red channel is replaced by NDVi and the green and blue are kept, so the model still has 3 inputs. The full dataset includes both RGB and NIR imagery, but authors use the default 3 channel DeepLabV3+ architecture, so they do not feed raw 4 channel RGB + NIR directly into the model. Instead, using NDVI as a workaround to incorporate NIR information directly.
5. Which model performs best and why, according to the authors?
   The best model is the NDVI-ML, the authors say it outperformed both deep learning methods on both Kimisala test datasets. They argue this happened because NDVI-ML does not require training data, while the deep learning models were hurt by the fact that the training and testing images differed in appearance even when captured with similar systems. Among the deep learning methods, the custom CNN with RGB + NIR beat DeepLabV3+ on Kimisala, and DeepLabV3+ with NDVI-GB was slightly better than DeepLabV3+ with RGB only.